

Projet de fin de semestre

Plateforme distribuée de détection et qualification d'e-mails de phishing

Énoncé de projet - Réalisable en 15 jours - Python uniquement

Pourquoi ce sujet ? Les campagnes de phishing assistées par IA et les techniques de social engineering restent parmi les risques les plus visibles du moment. Ce projet est donc à la fois actuel, concret et parfaitement compatible avec les notions du module : applications réparties, APIs, sérialisation, gestion des erreurs, RPC/objets distants et sécurité by design.

1. Contexte

Une organisation reçoit chaque jour des e-mails suspects : faux liens, demandes urgentes, usurpation d'identité, pièces jointes douteuses ou messages générés automatiquement pour tromper les utilisateurs. Elle souhaite mettre en place une mini plateforme distribuée capable de centraliser, analyser et qualifier ces signalements. L'objectif n'est pas de construire un antivirus complet, mais une application pédagogique, structurée et sécurisée.

2. Objectif général

Concevoir et développer en Python une mini application distribuée qui permet :

- l'authentification des utilisateurs
- la soumission d'un e-mail suspect sous forme de texte ou de métadonnées simplifiées
- l'analyse du contenu par un service dédié
- l'attribution d'un score de risque : faible, moyen ou élevé
- la consultation de l'historique des signalements
- la traçabilité des actions sensibles dans des logs d'audit

3. Travail demandé

Le projet doit comporter au minimum les composants suivants :

- AuthService : gère la connexion, la vérification d'un token simple et les rôles utilisateur / administrateur
- SubmissionService ou API Gateway : reçoit les signalements, valide les entrées et transmet les données aux autres services
- AnalysisService : analyse le message à l'aide de règles simples, d'un score heuristique ou d'un moteur léger
- AuditService : enregistre les événements de sécurité et les erreurs importantes
- Client : interface console, mini interface web ou script de test permettant d'utiliser la plateforme

4. Données minimales à gérer

- identifiant du signalement

- expéditeur déclaré
- objet de l'e-mail
- contenu textuel ou extrait
- liste éventuelle d'URLs détectées
- date de soumission
- utilisateur ayant soumis l'alerte
- score de risque
- justification courte du score

5. Fonctionnalités minimales obligatoires

Authentification et autorisation

- connexion avec login et mot de passe
- retour d'un token ou d'une session simplifiée
- refus des accès non authentifiés
- présence d'au moins deux rôles : administrateur et analyste ou utilisateur

Soumission et analyse

- soumettre un e-mail suspect
- vérifier et nettoyer les entrées côté serveur
- calculer un score de risque à partir de règles explicables
- retourner une décision lisible : faible, moyen, élevé

Consultation

- lister les signalements
- consulter le détail d'un signalement
- rechercher par expéditeur, score ou mot-clé

Audit et résilience

- générer des logs structurés
- ajouter au moins un timeout sur un appel distant
- gérer proprement l'indisponibilité d'un service
- renvoyer des erreurs non bavardes côté client

6. Contraintes techniques

- Python uniquement
- architecture répartie avec au moins 3 composants qui communiquent réellement
- échanges en JSON pour les appels API
- utilisation d'au moins un mécanisme RPC ou objet distant pour un sous-service, par exemple gRPC ou Pyro5
- validation stricte des entrées côté serveur
- journalisation structurée
- code organisé par services, modules et responsabilités
- aucune dépendance à une API payante ou à un service cloud obligatoire
- le projet doit rester démontrable localement sur une seule machine si nécessaire

7. Orientation de l'analyse

Le moteur d'analyse peut rester simple. Il n'est pas demandé de faire un modèle d'IA complet.

- détection de mots urgents ou manipulateurs
- détection de domaines suspects ou d'URLs inhabituelles
- écart entre l'adresse affichée et le domaine détecté
- présence de pièces jointes annoncées dans les métadonnées
- score cumulatif basé sur des règles
- explication textuelle des raisons du score

8. Exigences cybersécurité

- ne jamais stocker ni afficher les mots de passe en clair
- ne pas journaliser les tokens complets
- valider toutes les entrées côté serveur
- contrôler les rôles et les permissions
- prévoir des messages d'erreur génériques côté client
- prévoir une réflexion sur les risques de sérialisation et de désérialisation
- limiter la taille des entrées
- prévoir un minimum de protection contre l'abus d'appels
- documenter les menaces principales et les contre-mesures choisies

9. Architecture attendue

Une architecture simple mais cohérente est attendue. Exemple possible :

- Client -> API Gateway / SubmissionService
- API Gateway -> AuthService pour vérifier l'identité
- API Gateway -> AnalysisService pour le scoring
- API Gateway -> AuditService pour les événements sensibles
- Stockage local via SQLite ou fichiers JSON

Le projet peut être déployé localement en plusieurs processus. L'essentiel est de montrer une séparation claire des responsabilités, une communication distribuée réelle et une prise en compte de la sécurité.

10. Livrables

- code source complet
- README avec instructions d'installation et d'exécution
- schéma d'architecture
- rapport synthétique de 4 à 8 pages
- jeu de données de démonstration ou exemples d'e-mails suspects
- captures d'écran ou script de démonstration
- tableau des menaces principales et des protections mises en place

11. Démonstration attendue

- connexion d'un utilisateur
- soumission d'un e-mail suspect
- appel entre au moins deux services
- retour d'un score de risque
- refus d'un accès non autorisé
- exemple d'erreur gérée proprement
- exemple de log d'audit
- explication rapide d'un choix de sécurité

12. Plan de travail conseillé sur 15 jours

- Jours 1 à 3 : analyse du besoin, schéma de l'architecture, choix des services et répartition du travail
- Jours 4 à 6 : développement de l'authentification et de l'API de soumission
- Jours 7 à 9 : développement du service d'analyse et de la logique de scoring
- Jours 10 à 11 : intégration des logs, validation, gestion des erreurs et sécurité minimale
- Jours 12 à 13 : tests, correction et préparation des scénarios de démonstration
- Jours 14 à 15 : finalisation du rapport, nettoyage du code et répétition de la soutenance

13. Critères d'évaluation

- fonctionnement distribué et communication entre services : 30 %
- qualité technique et organisation du code : 25 %
- cybersécurité : validation, contrôle d'accès, gestion d'erreurs, logs : 25 %
- rapport, démonstration et justification des choix : 20 %

14. Bonus possibles

- tableau de bord web simple
- comparaison JSON vs Protobuf pour un échange précis
- mini système de file d'attente ou de traitement asynchrone
- circuit breaker simplifié
- versionnement d'un signalement ou historique des décisions
- analyse automatique d'URLs plus détaillée
- intégration d'un service RPC clairement séparé

15. Consigne finale

Le projet doit rester **réaliste, propre et démontrable**. Il vaut mieux une solution simple mais fonctionnelle, bien structurée et sécurisée, qu'une plateforme trop ambitieuse mais incomplète.